

课程笔记：Building Great Agent Skills: The Missing Manual

SSS & Codex

2026 年 7 月 1 日



视频作者/频道：AI Engineer

发布日期：2026 年 6 月 29 日

视频时长：20:43

视频链接：<https://www.youtube.com/watch?v=UNzCG3lw6O0>

PDF 制作 Skill: youtube-render-pdf

目录

1 引言：为什么会出现 skill hell	3
1.1 动机：先给出整场演讲的答案	3
1.2 核心观点：skill hell 是判断标准缺失	3
1.3 机制：技能失败会同时伤害个人和组织	3
1.4 例子：Skill Hell 这一页提示了真正问题	4
1.5 行动要点：先建立评审语言	5
1.6 本章小结	5
2 总览：一份检查好技能的 checklist	5
2.1 动机：没有 checklist，就无法区分好坏	5
2.2 核心观点：checklist 是技能质量的共同接口	6
2.3 机制：四个维度覆盖技能生命周期	6
2.4 例子：A Skill Checklist 这页就是全篇地图	7
2.5 行动要点：用清单做第一次快速评审	7
2.6 本章小结	8
3 Trigger：先决定技能由谁触发	8
3.1 触发方式决定成本由谁承担	8
3.2 context pointer：描述文本如何指向 skill.md	9
3.3 三种负载：context load、cognitive load、unpredictability	9
3.4 如何做触发决策	11
3.5 本章小结	11
4 Structure：把步骤和参考资料分清楚	11
4.1 结构问题：skill.md 容易把流程和材料混在一起	11
4.2 steps：把代理行为做成可执行流程	12
4.3 reference：把支持材料放在可查的位置	13
4.4 从零写一个 skill 的结构顺序	13
4.5 边界情况：只有 steps 或只有 reference 的 skill	14
4.6 本章小结	14
5 最小化 skill.md：用 context pointer 控制体积	14
5.1 为什么主文件要小	14
5.2 分支：判断参考资料是否属于主路径	15

5.3	context pointer: 把分支材料移出主文件	16
5.4	external reference: 外部参考仍属于 skill 包	17
5.5	隐藏分支参考材料	17
5.6	本章小结	18
6	Steering: 用 leading words 改变代理行为	18
6.1	问题: 技能写清楚了, 代理仍然可能跑偏	18
6.2	机制: 短语进入 reasoning tokens, 再反过来塑造行为	18
6.3	vertical slice: 一个压缩了工程经验的 leading word	19
6.4	如何挑选和使用 leading words	20
6.5	本章小结	21
7	Leg work: 让当前阶段做足工作	21
7.1	问题: 代理看见终点后, 会压缩前置步骤	21
7.2	机制: phase isolation 与 attention allocation	22
7.3	grill-with-docs 与 to-prd: 把计划前置工作拆成两个技能	23
7.4	什么时候值得隐藏未来步骤	24
7.5	本章小结	24
8	Pruning: 删除沉积物、杂质和无操作文本	24
8.1	为什么最后还要做 pruning	24
8.2	Massive skills: 巨大技能只是症状	25
8.3	Sediment: 多人协作留下的沉积物	26
8.4	Crud: 杂质会遮住真正的操作意图	27
8.5	No-ops: 看起来有意义, 实际不改行为	27
8.6	Deletion test: 用行为差异审查段落	29
8.7	本章小结	30
9	总结与延伸	30
9.1	演讲者的最后 checklist sweep	30
9.2	资源指针: writing great skills	31
9.3	作者提炼: 把 checklist 变成评审流程	32
9.4	四项主框架之间的内在关系	33
9.5	把这套方法用于真实团队	34
9.6	本章小结	34

1 引言：为什么会出现 skill hell

1.1 动机：先给出整场演讲的答案

这场演讲的核心答案可以压缩成一句话：一项好的 agent skill，需要用一份短小、可检查、能落地的 checklist 来设计和评审；这份清单依次检查 trigger、structure、steering 与 pruning。换成工程语言，它回答的是四个问题：技能何时进入 agent 上下文、技能内部如何组织、技能怎样改变 agent 行为、技能如何持续瘦身。

全篇主张

好技能的判断标准落在一个结果上：它能否稳定改变 agent 的下一步行为。评审一项技能时，先看四件事：触发是否清楚、结构是否可导航、行为引导是否有效、冗余内容是否被定期清理。

这就是本 PDF 后续章节的主线。演讲者把问题命名为 skill hell，意思是：技能越来越多，仓库越来越热闹，用户仍然难以判断哪一个技能值得信任，组织也难以把自己的操作规程改造成 agent 真正能执行的流程。

1.2 核心观点：skill hell 是判断标准缺失

skill hell 的危险核心是共同评审语言缺席，技能数量增长只是表面现象。一个团队可能已经下载了许多社区技能，也可能写了很多内部流程文档；然而只要没人能说清楚“这项技能好在哪里、坏在哪里、怎样改进”，技能库就会变成一堆互相挤占上下文的文本。

用一个直观类比来理解：学习编程时的 tutorial hell，通常表现为看了很多教程，却无法把知识拼成一个真实项目；前端生态里的 framework hell，表现为框架选择不断变化，开发者总被迫追逐下一个热门名词。skill hell 继承了这两类困境：资料很多、选择很多、承诺很多，实际可复用的行为改进却很少。

三个“hell”的共同结构

tutorial hell 像拿着一堆地图碎片却没有路线；framework hell 像每隔十分钟换一套交通规则；skill hell 像工具箱塞满了工具，却没人知道哪把扳手适合当前螺母。三者的共同根因是：缺少可迁移的判断标准。

1.3 机制：技能失败会同时伤害个人和组织

对个人而言，skill hell 会制造一种“再装一个技能就能解决”的幻觉。用户不断尝试新技能，agent 的表现仍然飘忽，最后问题被误判为模型能力不足。实际上，很多失败来自技能设计本身：触发太随意、结构太臃肿、指令无法改变行为、旧文本长期堆积。

对组织而言，问题更尖锐。组织真正想要的通常是把 operating procedures，也就是日常操作规程，变成 agent 可执行的工作方式。若技能只停留在“把流程文档复制进 skill.md”这个层面，agent 读到的是一堆文本，未必得到清晰的执行路径。组织因此拿不到技能承诺的收益：稳定复用经验、降低培训成本、让流程在多名 agent 或多名员工之间保持一致。

可以把这种风险写成一个简化关系：

$$R_{\text{skill hell}} \propto N_{\text{skills}} \times U_{\text{quality}} \times U_{\text{composition}}$$

- $R_{\text{skill hell}}$ ：技能地狱风险。
- N_{skills} ：可用技能数量。数量越大，选择和组合成本越高。
- U_{quality} ：质量不确定性。用户无法判断一项技能是否可靠。
- $U_{\text{composition}}$ ：组合不确定性。用户无法判断多项技能怎样一起工作。

这个公式是对前两分钟论点的工程化压缩，并非演讲中的数学模型：数量增长本身可以是好事；当质量和组合方式缺少评审框架时，增长会放大混乱。

1.4 例子：Skill Hell 这一页提示了真正问题

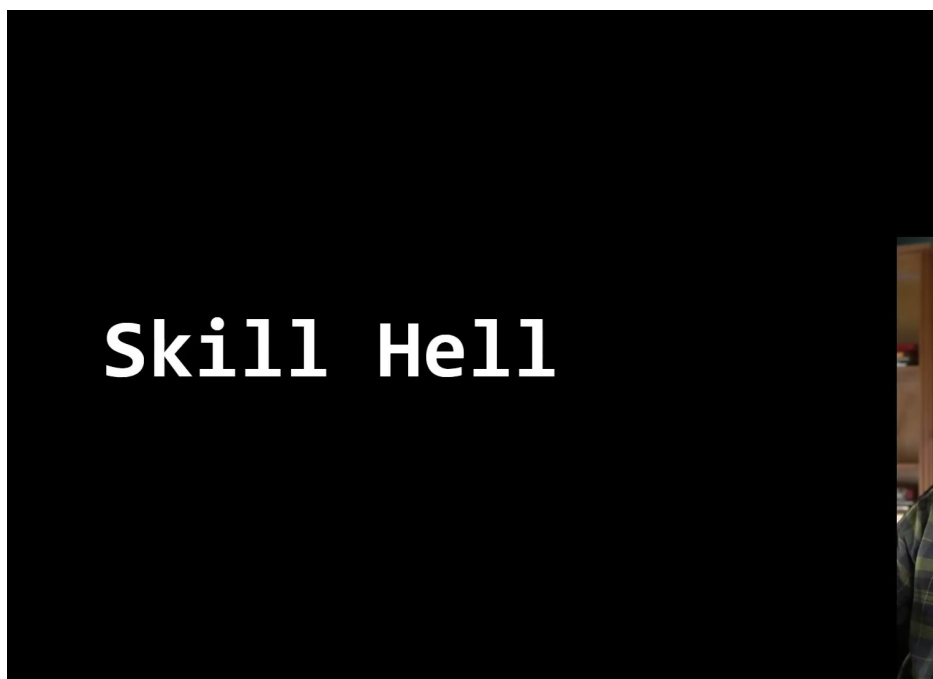


图 1: Skill Hell 这张标题页把问题先命名出来：技能生态进入爆发期后，用户最缺的是一套判断技能质量的语言，更多下载链接解决不了这个问题。¹

这张图本身很简洁，只有 Skill Hell 两个词。教学价值正来自这种简洁：演讲者先把一个模糊的行业感受命名出来，再把后续清单作为解决方案推出。命名很重要，因为没有名字的问题很难被讨论；有了 skill hell，团队才能把“这个技能不好用”进一步拆成“触发方式错了”“结构混了”“引导词无效”“沉积物太多”等可处理的问题。

¹视频画面时间区间：00:01:15–00:01:30。

skill hell 的真实风险

最危险的状态是团队以为自己已经拥有“技能体系”，实际只有一堆未经评审的 Markdown。技能一旦变成这种状态，就会给 agent 增加上下文负担，给用户增加选择负担，给组织增加维护负担。

1.5 行动要点：先建立评审语言

本章给读者的第一条实践建议很直接：在写新技能或引入社区技能前，先要求它接受同一套评审语言。最低限度应能回答以下问题：

- **trigger**：这项技能由用户手动触发，还是允许模型自己触发？
- **structure**：技能内部是否把步骤、参考资料、模板和分支清楚分开？
- **steering**：技能里的关键词是否真的会改变 agent 的计划、工具使用或输出？
- **pruning**：技能是否有清理 **sediment**、**crud** 与 **no-ops** 的机制？

1.6 本章小结

skill hell 指的是技能数量增长后，个人和组织仍然缺少判断、组合、改进技能的方法。演讲者的解决路线是建立一份可复用的 **skill checklist**。后续章节会沿着这份清单展开：先看触发，再看结构，然后看行为引导，最后看清理与瘦身。

2 总览：一份检查好技能的 checklist

2.1 动机：没有 checklist，就无法区分好坏

如果第一章的问题是 **skill hell**，第二章的答案就是 **skill checklist**。演讲者给出的清单承担两种用途：写新技能时，它像施工图；改旧技能时，它像体检表。它让团队从“感觉这个技能不太好”升级到“这个技能在触发、结构、引导或清理上具体哪里出了问题”。

四项检查清单

一项技能至少要过四关：

- **Trigger**：谁负责触发技能，触发成本落在模型还是用户身上？
- **Structure**：技能内部是否清楚区分步骤、参考资料和分支材料？
- **Steering**：技能是否用足够强的语言锚点让 agent 改变行为？
- **Pruning**：技能是否定期移走沉积、杂质和无操作文本？

2.2 核心观点：checklist 是技能质量的共同接口

这份 checklist 的作用类似代码评审里的共同接口。没有接口时，每个人只能凭经验说“这个实现怪怪的”；有了接口，评审者可以指向具体维度：触发规则会不会污染上下文，结构会不会让 agent 迷路，引导词会不会进入推理轨迹，冗余段落删掉后行为是否仍然一样。

它也让技能从“个人偏好”变成“可讨论的设计对象”。例如同一项技能可以有两种触发策略：`user-invoked skill` 由用户显式调用，能降低模型上下文负载；`model-invoked skill` 允许 agent 根据 `description` 自行调用，能降低用户记忆成本。哪一种更合适，取决于任务频率、误触发代价、上下文预算和用户熟练度。

2.3 机制：四个维度覆盖技能生命周期

这四项检查有清晰的生命周期顺序，覆盖技能从进入上下文到长期维护的完整过程。

1. **Trigger 决定入口**。入口设计错误时，技能要么进不来，要么进得太频繁。前者让好技能沉睡，后者让上下文被大量 `description` 占满。
2. **Structure 决定可读性**。结构好的技能会把 `steps` 和 `reference` 分开，使 agent 知道当前要执行什么，遇到模板或背景材料时去哪里查。
3. **Steering 决定行为改变**。技能文本只有被 agent 在计划、推理和输出中反复使用，才算真正发挥作用。后文会重点讲 `leading words`，也就是压缩行为意图的引导词。
4. **Pruning 决定长期健康**。技能一旦多人维护，就会积累 `sediment`、`crud` 和 `no-ops`。清理机制缺席时，技能会越来越大，效果却越来越难验证。

从工程角度看，四项检查可以理解为一流水线：

$$\text{usable skill} = f(\text{trigger}, \text{structure}, \text{steering}, \text{pruning})$$

- `trigger` 控制技能是否在正确时刻出现。
- `structure` 控制技能进入上下文后是否容易执行。
- `steering` 控制技能是否能改变 agent 的实际行为。
- `pruning` 控制技能在多轮修改后是否仍然清楚、短小、有效。

2.4 例子：A Skill Checklist 这页就是全篇地图

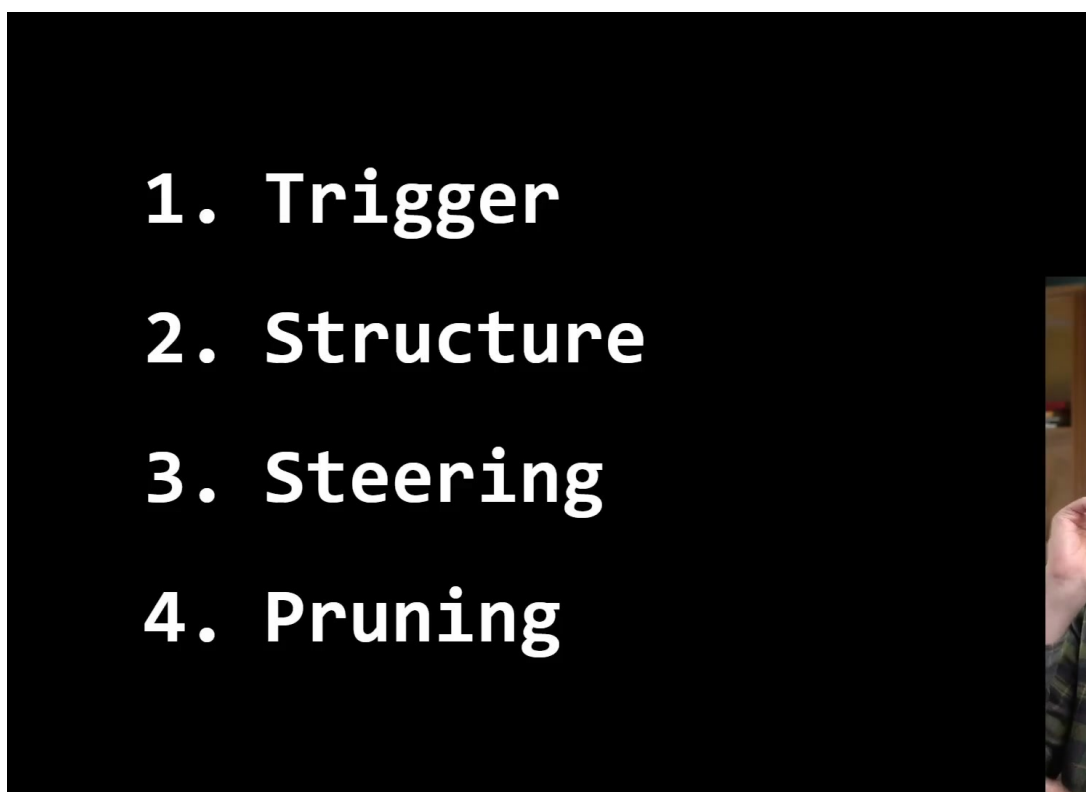


图 2: 演讲者用这一页给出全篇地图: Trigger、Structure、Steering、Pruning。后续章节依次展开每一项。²

这张图适合放在总览章，因为它把所有后续内容压缩到四个词。读者可以把它当成索引：看到某个技能调用混乱，就回到 Trigger；看到 skill.md 又长又乱，就回到 Structure 和 Pruning；看到 agent 明明读了技能仍然按老习惯行动，就回到 Steering。

checklist 与 rubric 的区别

checklist 更像逐项确认的清单，适合防止漏项；rubric 更像评分标准，适合比较多个候选方案。本演讲里的清单兼有两种功能：它既提醒作者别漏掉关键设计面，也帮助团队评价一项技能的质量。

2.5 行动要点：用清单做第一次快速评审

读者拿到一项现有技能时，可以先做一个五分钟快速评审：

- 读 description，判断它会不会让模型过度或错误触发。
- 扫 skill.md，标出步骤、参考资料、模板、分支说明是否混在一起。
- 找关键词，看它们是否足够短、稳定、可复用，能成为后文所说的 leading words。
- 做一次概念上的删除测试：某段文字拿掉后，agent 行为是否会发生变化。

²视频画面时间区间：00:02:25–00:02:40。

清单使用中的误区

这份 `checklist` 不能替代实际运行。它先帮助读者发现高风险结构，再通过真实任务验证技能是否改变行为。只看文字漂亮程度，很容易把长篇说明误判成高质量技能。

2.6 本章小结

`skill checklist` 是走出 `skill hell` 的第一层工具。它把技能质量拆成四个可检查维度：入口、结构、行为、清理。接下来，PDF 会从 `Trigger` 开始，把每一项拆成具体机制、常见代价和可执行的改进方式。

3 Trigger：先决定技能由谁触发

3.1 触发方式决定成本由谁承担

一个 `skill` 进入代理工作流之前，先要回答一个很朴素的问题：它由谁触发？视频在这里用 `mattpocock/skills` 与 `obra/superpowers` 的对照开场，核心问题是 `skill` 的第一层设计约束：`trigger` 会把成本分配给模型、用户和评估流程。

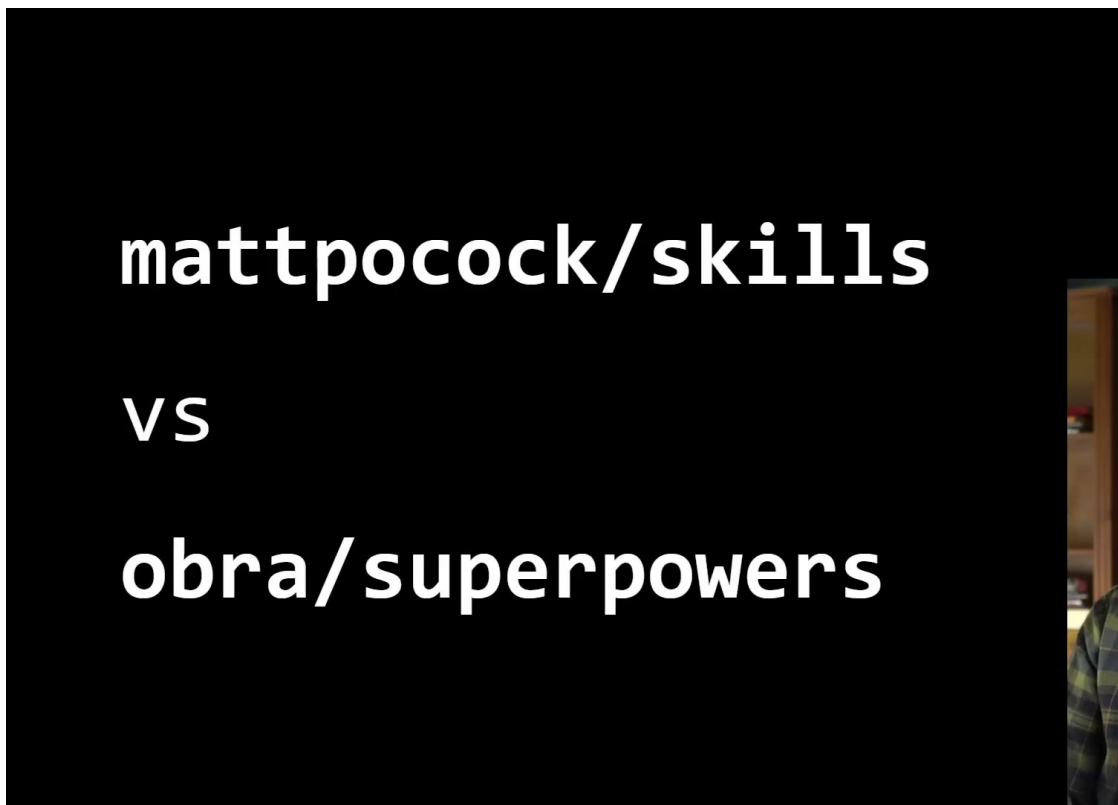


图 3: `mattpocock/skills` 与 `obra/superpowers` 的对照引出 `trigger` 问题：skill 的触发机制会改变整套使用体验。³

³视频画面时间区间：00:03:25–00:03:40。

`user-invoked skill` 指用户显式调用的技能。它像工具箱里的专用扳手，平时安静地躺在文件系统中，用户需要它时才告诉代理去读它。这样的好处是代理的默认上下文更干净；代价是用户必须知道有这把扳手，知道什么时候拿出来。

`model-invoked skill` 指代理可以根据 `description` 自行决定调用的技能。它像车里的自动驾驶辅助：用户少做选择，系统会尝试自行判断场景。它的代价也随之出现：每个可自动触发的技能通常要把 `description` 放进上下文，代理还要决定是否沿着这个线索读取 `skill.md`。

Trigger 的核心判断

`trigger` 设计本质上是在分配三类成本：模型要承受多少 `context load`，用户要承受多少 `cognitive load`，系统要承受多少 `unpredictability`。一个 `skill` 的触发方式越自动，用户越省心；同时代理上下文越拥挤，触发行为越需要被验证。

3.2 context pointer：描述文本如何指向 skill.md

视频把 `description` 解释成一种 `context pointer`。这个词很重要：它承担路牌职责。路牌写着“这里可能有你需要的材料”，真正的材料通常在 `skill.md` 里。模型看到路牌后，可以选择沿着它读取更多上下文。

这个机制能解释两类 `skill` 的差异。若 `skill` 允许模型触发，`description` 会进入代理上下文；若 `skill` 只允许用户触发，这块路牌可以对模型隐藏，只展示给用户。视频里的例子是 `grill me` 一类技能使用了类似 `disable model invocation true` 的设置，让描述文本不会常驻模型上下文。

```
1 ---
2 name: grill-me
3 description: Ask focused clarification questions before planning.
4 disable_model_invocation: true
5 ---
```

Listing 1: Minimal user-invoked skill metadata

这个小配置表达的是一个设计意图：技能仍然存在，用户仍然能调用它；代理在普通请求里不会因为这条描述而增加额外判断负担。对大型 `skill` 集合来说，这个差别会被放大。10 个自动触发技能还容易承受，100 个自动触发技能意味着 100 条描述在每轮请求里参与竞争。

直觉类比：路牌、地图和驾驶员

`context pointer` 像路牌，`skill.md` 像详细地图，代理像驾驶员。把路牌放进上下文，驾驶员有机会发现那张地图；路牌太多时，驾驶员会分心；路牌被隐藏时，只有用户主动指出路线，驾驶员才会去找那张地图。

3.3 三种负载：context load、cognitive load、unpredictability

选择触发方式时，视频给出的关键提醒很现实：更自动的 `skill` 并不免费。每个 `model-invoked skill` 都会增加 `context load`。这里的 `context load` 包含两层含义：一层是 `token` 成本，每条描述都占用上下文窗口；另一层是注意力成本，模型需要在更多候选技能之间判断。

`user-invoked skill` 把另一类成本转给用户，也就是 `cognitive load`。用户需要记住技能名称、适用场景、调用方式和边界条件。用户越熟悉 `skill` 库，这个成本越低；新手面对一大堆手动技能时，很容易陷入“知道有工具，却不知道什么时候用”的状态。

第三个成本是 `unpredictability`。只要触发权交给模型，系统就会出现一个新的不确定点：模型可能在非常适合的任务中忽略某个 `context pointer`，也可能在边界场景里调用了低相关技能。于是团队往往需要额外做 `eval`，检查技能是否在正确时间被调用。这类评估成本容易被低估，因为它不会写在 `skill.md` 里，却会出现在维护周期里。

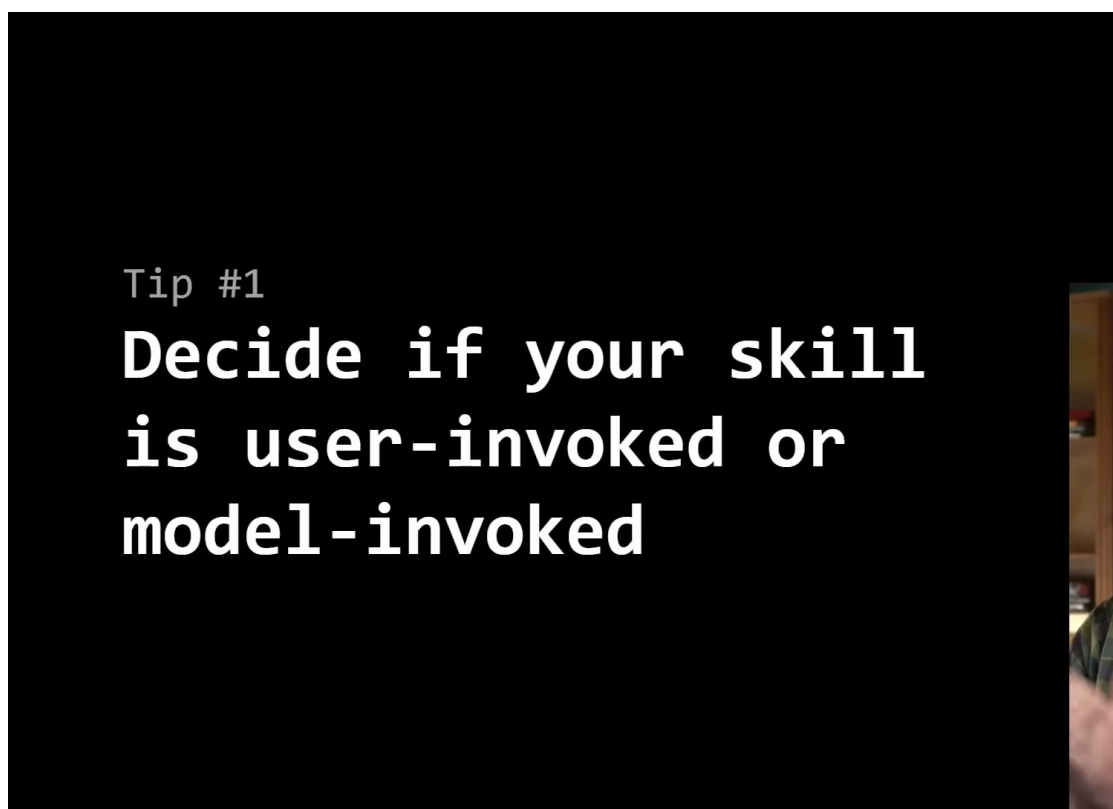


图 4: 视频中的 Tip #1: 先决定 skill 是 `user-invoked` 还是 `model-invoked`。⁴

可以把这一段压缩成一个近似成本公式。这个公式属于工程近似，作用是一张检查表：每增加或减少一个 `skill`，设计者都要问三项成本由谁承担、如何验证、是否值得。

$$C_{\text{skill}} \approx C_{\text{context}} + C_{\text{cognitive}} + C_{\text{unpredictability}}$$

- C_{skill} : 一个 `skill` 在真实使用中的总成本，包括运行、理解、维护和评估。
- C_{context} : 代理上下文负载，主要来自常驻描述、额外 `token`、候选技能竞争和注意力分散。
- $C_{\text{cognitive}}$: 用户认知负载，主要来自记忆技能、判断适用场景、手动调用和理解失败原因。
- $C_{\text{unpredictability}}$: 不确定性成本，主要来自模型是否会跟随 `context pointer`、何时调用 `skill`、调用后是否走对流程。

⁴视频画面时间区间：00:05:05–00:05:20。

3.4 如何做触发决策

视频中的 Matt 倾向于让许多技能保持 `user-invoked`。这能保持模型上下文较小，也能减少“模型该调用却没调用”的问题。这个选择适合技能作者本人或熟练用户，因为他们愿意承担更多 `cognitive load`，换取更可控的调用路径。

团队环境会更复杂。若一个 skill 是高频、低风险、场景边界清楚的流程，例如固定格式的日志摘要、标准化安全检查、例行代码审查入口，`model-invoked skill` 可能更有价值。若一个 skill 是高风险、低频、需要用户明确授权或容易误触发的流程，例如发布、迁移、删除、付费操作、外部写入，`user-invoked skill` 更容易审计。

容易漏掉的风险：自动触发会制造评估义务

一旦允许模型自行触发 skill，团队就需要回答两个问题：该调用时它是否调用？不该调用时它是否克制？这两个问题通常需要 `eval` 或回放日志支撑。缺少这层验证时，自动触发会变成隐藏的不确定性来源。

3.5 本章小结

本章说明了 skill checklist 的第一项：`trigger`。一个 skill 可以由用户显式调用，也可以由模型根据 `description` 自动调用。前者降低 `context load`，提高 `cognitive load`；后者降低用户操作负担，增加上下文成本和 `unpredictability`。设计触发方式时，关键问题是成本分配；单一触发模式本身没有通吃价值。下一章会进入 skill 内部结构，讨论如何把 `steps` 和 `reference` 分清楚。

4 Structure：把步骤和参考资料分清楚

4.1 结构问题：skill.md 容易把流程和材料混在一起

确定 `trigger` 之后，第二个问题是 `structure`：skill 内部应该怎样组织。视频给出的最小结构非常实用：多数 skill 可以拆成两类单元，`steps` 和 `reference`。

`steps` 是代理要执行的步骤，强调顺序、动作和检查点。它回答“先做什么、再做什么、什么时候停下来”。`reference` 是执行步骤时需要查阅的支持材料，例如术语定义、模板、评估标准、示例、命名规范、写作契约。它回答“执行这一步时需要哪些背景材料”。

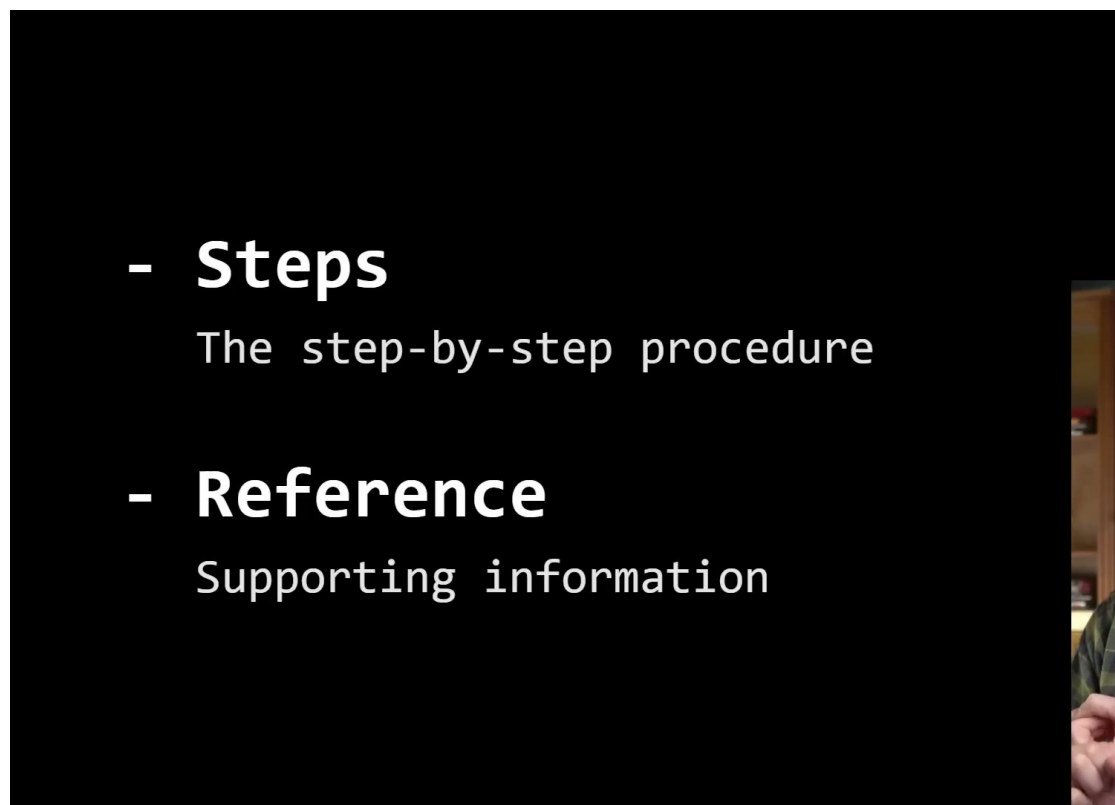


图 5: 视频把 skill 内部结构压缩成两个基本单元: `steps` 是流程, `reference` 是支持信息。⁵

结构的最小原则

写 skill 时先列 `steps`, 再问每一步需要哪些 `reference`。流程和材料分开后, skill 才容易审查、复用、裁剪和扩展。

4.2 steps: 把代理行为做成可执行流程

`steps` 的价值在于减少代理自由发挥的空间。一个好的步骤通常包含动作、顺序和完成条件。例如视频里的 `to-prd` 技能, 目标是从当前上下文生成 PRD。它把“写一份 PRD”拆成三个可执行动作:

1. 查找相关上下文。
2. 与用户确认 `test seams`。
3. 写出 PRD。

`test seam` 可以理解为“测试接缝”: 系统里最适合插入测试、验证行为或观察结果的边界。比如一个服务函数、一个 API 入口、一个数据转换层, 都可能成为测试接缝。视频强调这个人类确认点, 因为测试边界一旦选错, 后面的 PRD 会把错误验证路径写得很正式, 最终增加返工。

⁵ 视频画面时间区间: 00:07:35–00:07:50。

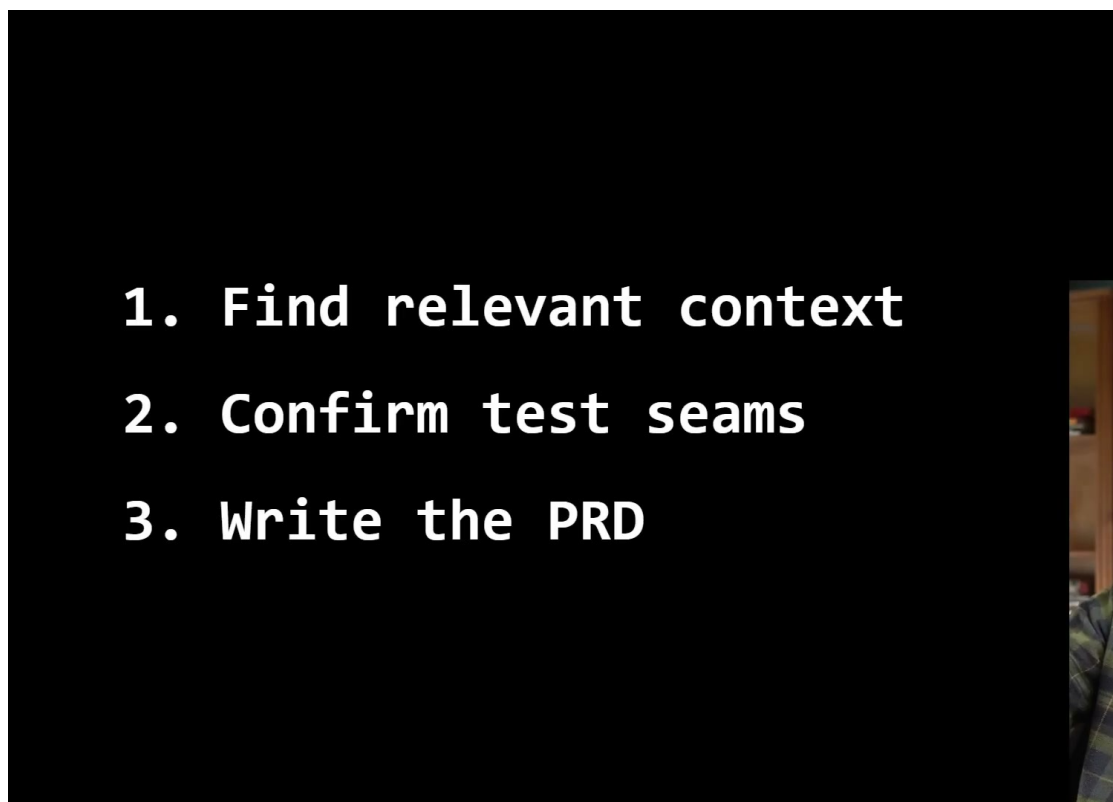


图 6: to-prd 的三步流程：找上下文、确认 test seams、写 PRD。⁶

4.3 reference：把支持材料放在可查的位置

reference 的职责是支撑步骤，需要避免抢走步骤的位置。以 to-prd 为例，它至少需要两类参考资料：第一，什么是 test seam；第二，PRD 的 Markdown 模板。前者帮助代理和用户围绕测试边界达成一致；后者保证最终产物结构稳定。

这类材料很容易被误放进流程正文里。误放之后，skill 会出现两个问题：一是步骤变长，代理难以看清当前动作；二是参考材料难以复用，别的步骤或分支想用同一模板时，只能复制一份。复制一多，后面就会出现版本漂移。

互补关系：steps 像菜谱步骤，reference 像食材说明

菜谱步骤告诉厨师“先预热烤箱，再混合面糊，再烘烤 25 分钟”；食材说明解释面粉类型、黄油软化状态和模具尺寸。把两者混在一起，短菜谱还能读；复杂菜谱会迅速失控。skill 也是同样的结构问题。

4.4 从零写一个 skill 的结构顺序

视频给出了一条从零写 skill 的顺序：先判断是否需要步骤，再写步骤，然后为步骤配参考资料。这个顺序值得遵守，因为它能避免“材料先行”的常见失败。很多弱 skill 会先堆满模板、术语、例子和背景说明，最后才补一句“请照此执行”。代理读到这种文件时，会获得大量信息，却没有清晰

⁶视频画面时间区间：00:08:15–00:08:30。

的行动轨道。

更稳的写法可以压缩成三问：

1. 这个 skill 是否需要固定流程？
2. 每个步骤的完成条件是什么？
3. 哪些参考资料只在某一步或某个分支中需要？

结构检查

若一段文字会改变代理的下一步动作，它更可能属于 **steps**；若一段文字只在执行某一步时提供定义、模板或判断标准，它更可能属于 **reference**。

4.5 边界情况：只有 steps 或只有 reference 的 skill

视频也提醒，skill 不一定同时拥有两类单元。有些 skill 几乎全是 **reference**，例如术语表、风格指南、审查标准；它们提供材料，动作流程很少。有些 skill 几乎全是 **steps**，例如一个很短的固定检查流程；它只要求代理依次执行几件事，几乎不需要额外资料。

真正要避免的是边界模糊。若一个 skill 既有流程又有大量材料，却没有把 **steps** 与 **reference** 分层，后续维护者会很难判断新增内容应放在哪里。长期看，这会直接推高后面章节要讨论的 `skill.md` 体积问题。

结构失控的早期信号

当维护者无法回答“这一段是让代理做事，还是让代理查资料”时，skill 已经开始变得难以审计。继续往同一个 `skill.md` 里追加材料，只会让步骤更难被看见。

4.6 本章小结

本章说明了 skill checklist 的第二项：**structure**。skill 内部最重要的区分是 **steps** 与 **reference**：前者规定代理行动，后者提供支持材料。视频用 `to-prd` 展示了一个清晰例子：找上下文、确认 `test seams`、写 PRD 是步骤；`test seam` 说明和 PRD 模板是参考资料。这个区分为下一章铺路：一旦能分清流程和材料，就能进一步控制 `skill.md` 的体积。

5 最小化 skill.md：用 context pointer 控制体积

5.1 为什么主文件要小

视频进入第三个结构约束：让 `skill.md` 尽量小。这里的“小”指主文件只承载所有使用路径都需要的核心指令，内容贫乏和主文件精简是两回事。每个 skill 通常由三层组成：**description**、`skill.md`、以及从主文件分出去的参考材料。主文件越短，维护者越容易审计，代理越容易抓住意图，token 成本也越低。

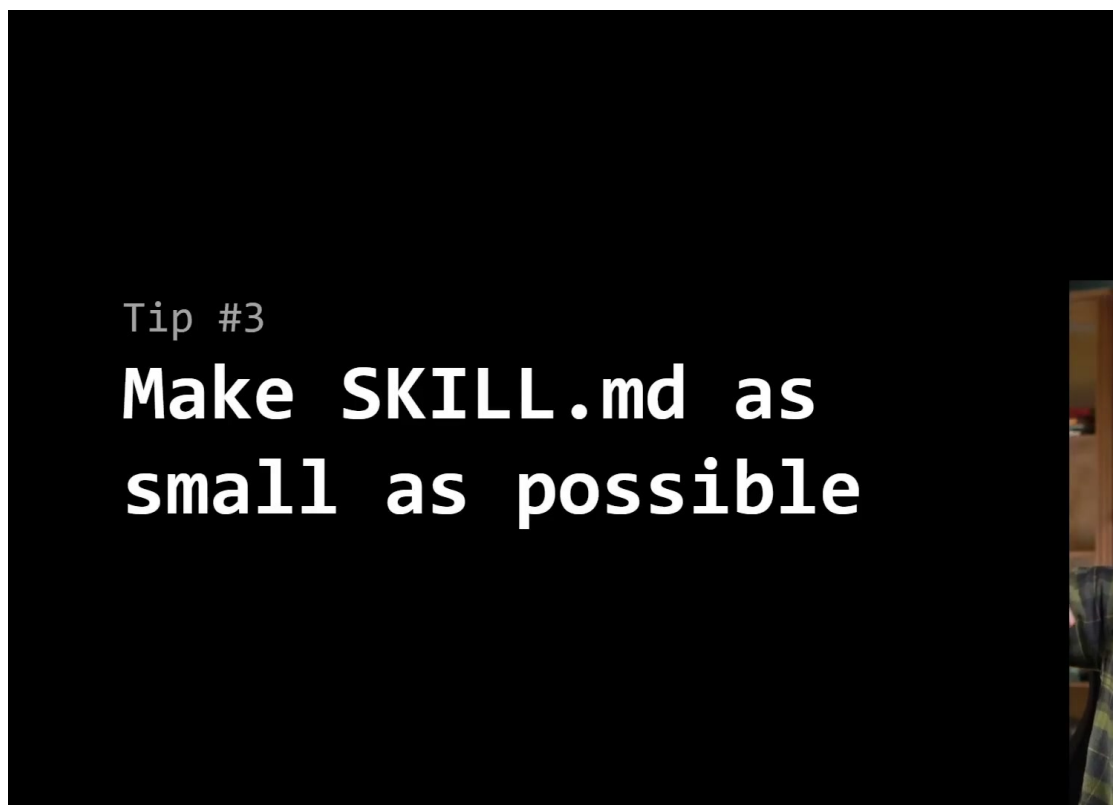


图 7: 视频中的 Tip #3: 让 SKILL.md 尽量小, 减少维护、审计和 token 负担。⁷

最小化的真正目标

`skill.md` 应该像机场中转大厅: 所有旅客都会经过这里, 所以只放共同规则、分流标志和必要检查点。只服务某一条航线的材料, 应放到那条航线对应的外部参考文件里。

5.2 分支: 判断参考资料是否属于主路径

让 `skill.md` 变小的关键, 是识别 `skill` 的分支。分支指同一个 `skill` 的不同使用路径。若某份 `reference` 每次都会用到, 它可以留在主文件附近; 若它只服务某个分支, 它就是迁出主文件的候选。

视频用 `to-prd` 做了第一种情况。这个 `skill` 每次都要写 PRD, 也每次都要确认 `test seams`。因此 PRD 模板和 `test seam` 解释很可能属于主路径材料。它们虽然是 `reference`, 但几乎每次运行都会需要。

第二个例子是 `domain modeling`。它可能更新本地词汇表 `context.md`, 也可能创建 ADR, 有时还可能两者都不做。这里出现了至少两条分支: 词汇表分支和架构决策记录分支。`context.md` 模板只服务词汇表分支, ADR 模板只服务决策记录分支。把两个模板都塞进主 `skill.md`, 会让每次调用都携带当前任务可能用不到的材料。

⁷ 视频画面时间区间: 00:09:05–00:09:20。

相关概念：ADR 与 context.md

ADR 是 Architecture Decision Record，即架构决策记录，用来记录某个架构选择的背景、决策和后果。context.md 在视频语境里是本地领域词汇表或上下文文档。二者都很重要，但它们服务不同分支，适合被放在各自的外部参考文件中。

5.3 context pointer：把分支材料移出主文件

context pointer 是主文件变小的基本工具。它在 skill.md 中保留一个短指针，告诉代理在需要某类材料时去读哪个文件。这样主文件只承担分流职责，详细模板和长参考资料留在外部。

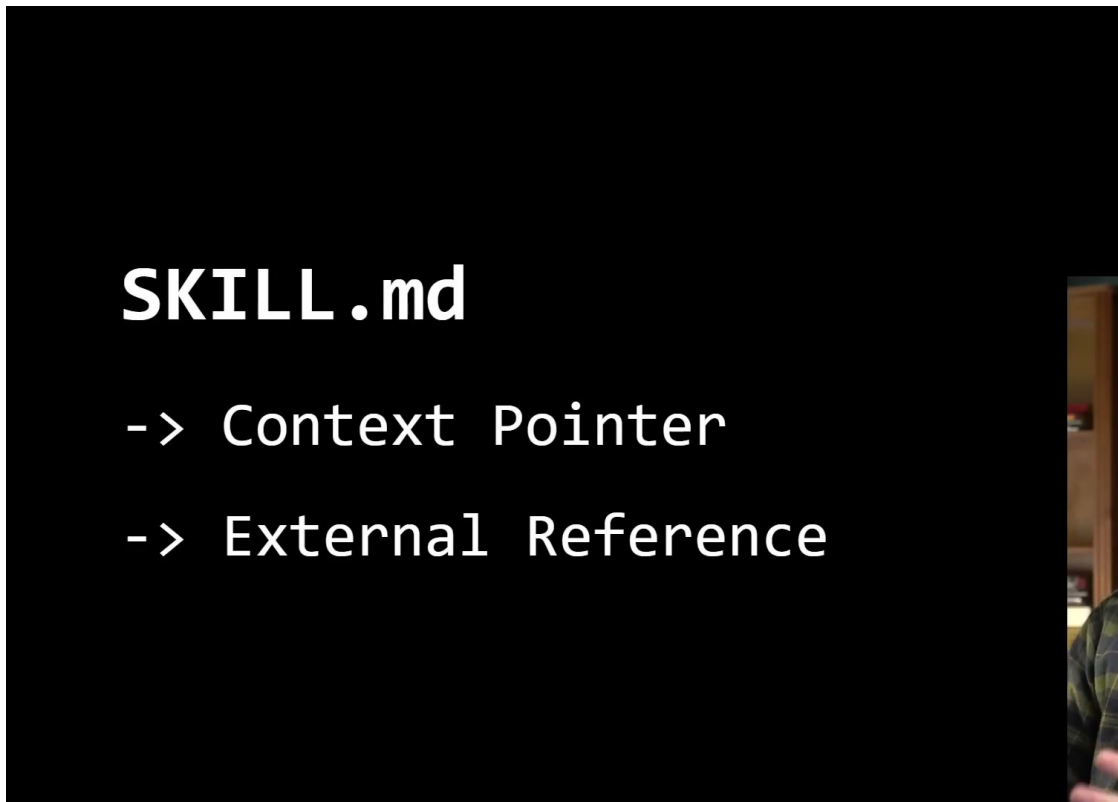


图 8: SKILL.md 通过 Context Pointer 指向 External Reference，把长材料移出主文件。⁸

一个极简指针可以写成下面这样。它不展开模板内容，只告诉代理在进入特定分支时加载对应参考文件。

```
1 When updating context.md:
2   Read references/context-template.md first.
3
4 When creating an ADR:
5   Read references/adr-template.md first.
```

Listing 2: Context pointer to branch-specific references

这段配置的关键在于延迟 context load。代理只有在进入某个分支时才读取对应模板。若当前

⁸视频画面时间区间：00:11:15–00:11:25。

任务只需要更新 `context.md`，它就不必携带 ADR 模板；若当前任务只需要创建 ADR，它也不必读取词汇表模板。

5.4 external reference：外部参考仍属于 skill 包

视频把这种被指向的文件称为 `external reference`。这里的 `external` 指它位于 `skill.md` 之外，并不表示它脱离 skill 包。它仍然可以随 skill 一起分发、版本控制和审查。这个细节很重要：若参考资料散落在网页、个人笔记或远程仓库里，代理运行时会多出网络、权限和版本漂移风险；若它们被打包在 skill 文件夹中，指针就更稳定。

容易忽视的风险：指针太多也会变成迷宫

`context pointer` 能降低主文件体积，也会增加跳转层级。若一个 `skill.md` 只剩一串文件路径，代理和维护者都要不断追踪材料来源。好的指针应短、准、少，并且每个指针都对应真实分支。

5.5 隐藏分支参考材料

视频用 Tip #4 总结了这一章的操作：把分支专用 `reference` 隐藏在 `context pointer` 后面。这里的“隐藏”指默认不加载，材料仍然可见。它更像城市路标：主干道只显示关键方向，详细街区地图放在对应出口之后。

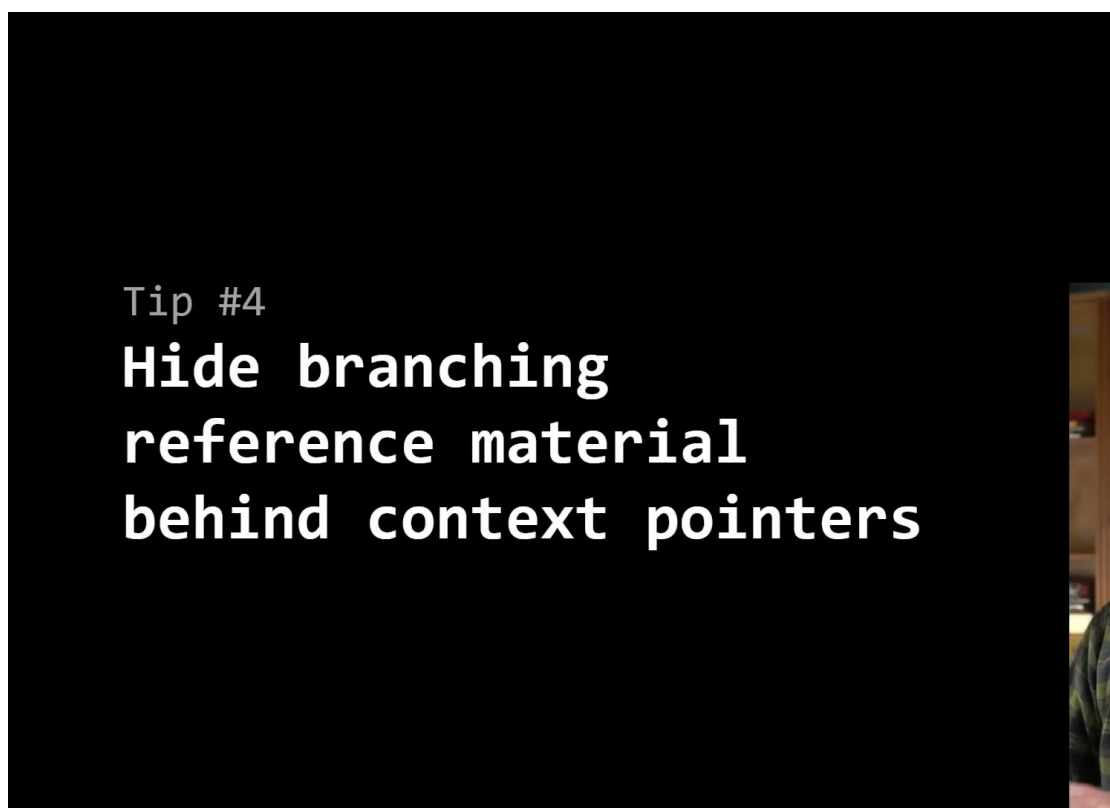


图 9: Tip #4: 把分支专用参考材料放在 `context pointers` 后面，按需加载。⁹

⁹ 视频画面时间区间：00:11:25–00:11:40。

判断一份材料是否应该迁出主文件，可以用三问：

1. 它是否只在某个分支使用？
2. 它是否占据大量 token 或注意力？
3. 它是否可以用一句明确指针稳定定位？

三项都成立时，它很适合成为外部 **reference**。若材料每次都会使用，或者没有它代理无法理解主流程，它更适合留在 `skill.md` 或紧贴主流程的位置。

5.6 本章小结

本章说明了如何最小化 `skill.md`。主文件应保留共同规则、核心 **steps** 和必要分流，分支专用 **reference** 应通过 **context pointer** 指向外部文件。视频中的 `to-prd` 说明了单分支 **skill** 的材料可以留得更近；**domain modeling** 说明了多分支 **skill** 应把 `context.md` 模板和 **ADR** 模板拆出去。这样做能降低 **context load**，减少维护者的审计负担，也让后续的 **steering** 更容易发挥作用。

6 Steering：用 leading words 改变代理行为

6.1 问题：技能写清楚了，代理仍然可能跑偏

前面几章解决了技能的入口和结构：什么时候触发、主文件多大、哪些材料放进 `skill.md`，哪些材料放到外部 **reference**。到了 **steering**，问题变得更尖锐：技能文本已经进入上下文，代理也读到了步骤，可它仍然可能没有按作者期望行动。

这类失败很常见。作者以一句话概括它：人觉得自己已经说明白了，代理执行时仍然没有做那件事。对人类读者而言，一段长说明可能足够清楚；对代理而言，长说明会被拆散到上下文、操作过程、计划草稿和输出里。真正能持续影响行为的，往往是短、稳、可复用的表达。

本节核心机制：leading words

leading words 是压缩行为意图的短语。它的价值在于：短语先出现在技能文本中，再被代理带入 **reasoning tokens**、计划语言和对用户的输出中；当这个短语不断被重复，代理就更容易按这个短语代表的行为模式行动。

6.2 机制：短语进入 reasoning tokens，再反过来塑造行为

leading words 可以理解为一种行为锚词。它像代码里的函数名：一个好函数名把调用者引向正确的抽象层次；一个好 **leading word** 把代理引向正确的工作姿态。短语越稳定，代理越容易把它当作当前任务的操作标签。

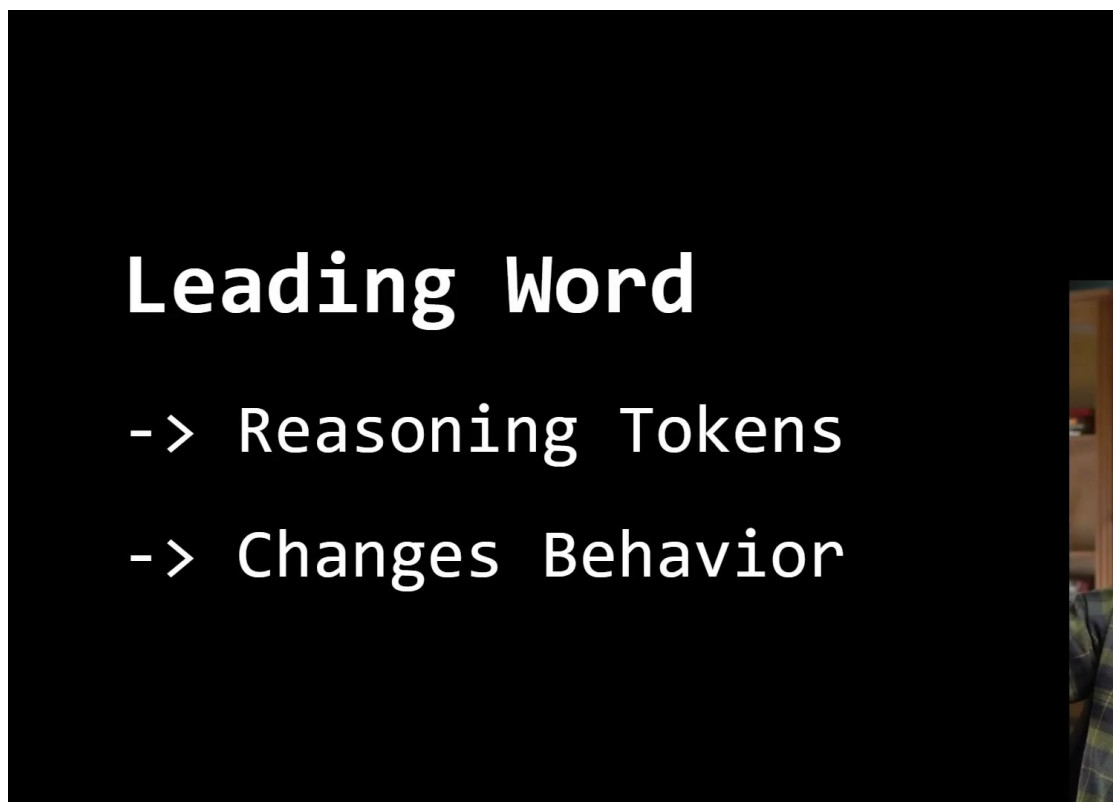


图 10: leading word 的基本链路：短语进入 reasoning tokens，随后改变行为。¹⁰

这条链路的关键在于反馈。技能作者写下一个短语，代理在内部操作和外部计划中复述它；复述让短语成为当前工作的一部分；当前工作再被这个短语约束。用一个粗略模型表示：

$$B_{\text{agent}} \approx f(I_{\text{skill}}, W_{\text{lead}}, R_{\text{trace}})$$

其中：

- B_{agent} 表示代理最终表现出来的行为；
- I_{skill} 表示技能中的一般指令；
- W_{lead} 表示反复出现的 leading words；
- R_{trace} 表示代理在推理、计划、阶段性输出中形成的语言轨迹。

这个公式的用途在于提醒读者：技能并非只靠完整句子生效。对代理来说，高频、低歧义、带工程传统的短语会改变注意力分配。它们像路标，反复告诉代理当前任务应沿哪条路径前进。

6.3 vertical slice：一个压缩了工程经验的 leading word

作者给出的核心例子是 vertical slice。典型代理在接到一个大实现任务后，容易按层推进：先写数据库层，再写 schema，再写 API，再写前端。这种推进方式看起来整齐，风险却很高，因为用户很晚才看到可运行的东西，反馈也被推迟。

¹⁰ 视频时间：00:12:35–00:12:50。

`vertical slice` 把另一种工作方式压缩成两个词：先做一条穿过各层的小闭环。它可以包含一点数据模型、一点接口、一点 UI、一点验证，目标是尽早让一个薄功能跑起来。相比逐层铺开，`vertical slice` 更强调早反馈、早集成和可观察进度。

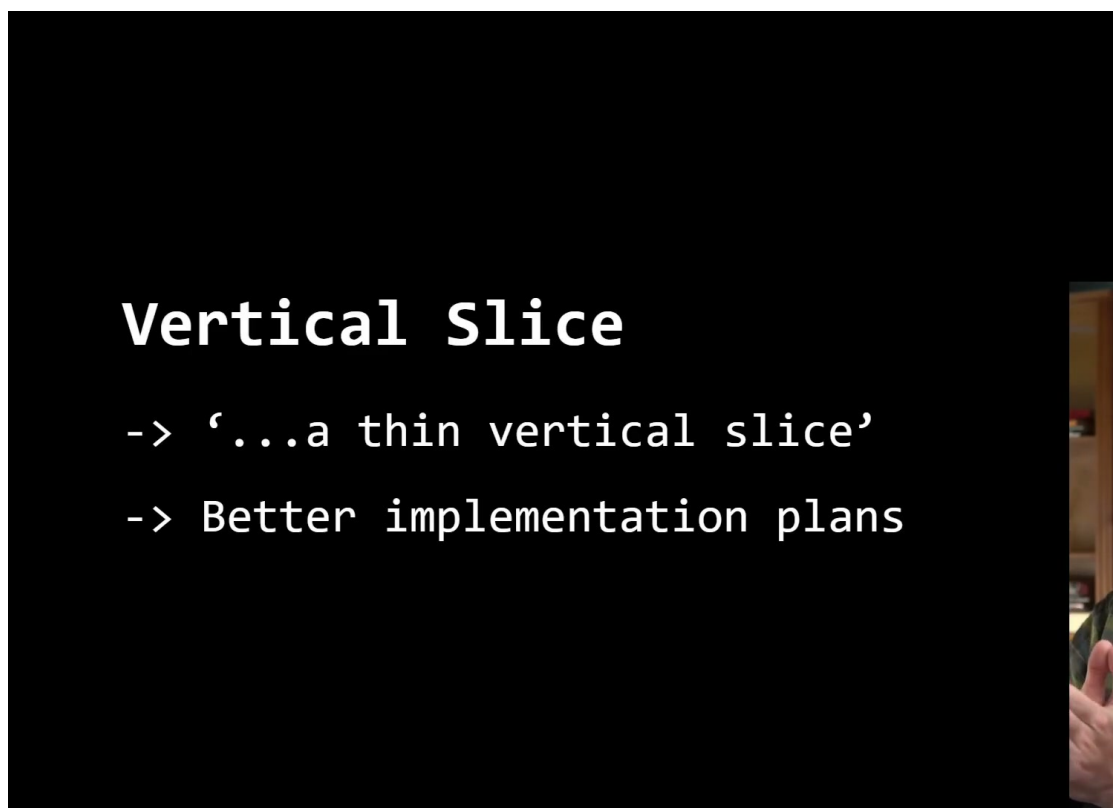


图 11: `vertical slice` 把“先做薄闭环”的工程习惯压缩成可重复的引导词，并帮助代理产出更好的 implementation plans。¹¹

为什么 `vertical slice` 有效

`vertical slice` 背后有成熟的软件工程语境。代理在训练中见过这个词的常见用法，因此它触发的是一整套先做端到端薄切片、尽早集成、避免层状堆积的实践模式。

这也解释了作者为什么提醒读者观察 `reasoning tokens`。如果技能里写了 `vertical slice`，而代理在计划里开始说“先做一个 thin vertical slice”，说明这个词已经进入了它的工作语言。此时，技能作者获得了一个可检查的信号：`leading word` 已经被代理吸收。若代理完全没有复述或围绕它组织计划，短语可能需要更强、更一致，或者需要在步骤中更明确地连接到行为。

6.4 如何挑选和使用 `leading words`

挑选 `leading words` 需要看三个条件。第一，短语要短，足以在技能里多次出现；第二，短语要具体，能指向可观察行为；第三，短语最好带有已有语境，让代理可以调用它的先验知识。作者把英语形容成很宽的 API，这个比喻很准确：不同短语像不同函数名，调用出来的行为模式会有差异。

¹¹ 视频时间：00:14:05–00:14:20。

弱 leading words 的常见问题

“认真做”“高质量完成”“充分考虑”这类表达很容易变成空泛提醒。它们表达愿望，却缺少具体行为形状。更强的短语通常能回答：代理下一步要怎样拆分任务、怎样取证、怎样验证、怎样避免某类失败。

因此，技能作者可以把短语当作可测试工具。写入技能后，观察代理是否在计划中复述它，是否用它选择任务切分方式，是否因此提前寻求反馈或改变实现顺序。一个真正有效的 leading word 会从文本标签变成行为约束。

6.5 本章小结

Steering 的核心在于用短语控制行为，而 leading words 是这一章的主机制。它们把复杂意图压缩为小而稳定的语言单位，再通过 reasoning tokens 和计划输出反复进入代理的工作过程。vertical slice 展示了这种机制的威力：两个词可以把“先做端到端薄闭环、尽早拿反馈”的工程习惯带进代理的实现计划。评审一个技能时，作者建议直接看代理是否采用了这些词；如果没有，技能文本对行为的 steering 仍然偏弱。

7 Leg work：让当前阶段做足工作

7.1 问题：代理看见终点后，会压缩前置步骤

leading words 解决的是“怎样把行为意图压缩进语言”的问题；leg work 解决的是另一个现实问题：代理在某个步骤上投入的实际工作量常常不足。这里的 leg work 指当前阶段应完成的扎实前置工作，例如追问需求、阅读文档、探索代码库、核对约束、确认测试边界。

作者指出，一个典型失败场景是 Plan Mode。许多 Plan Mode 都包含两个步骤：先问澄清问题，再创建计划。表面上这很合理；实际运行时，代理经常只问一两个问题，然后急着产出计划。原因很直接：它已经看到了终点。只要“最终要交付计划”这个目标在上下文中很显眼，前面的澄清阶段就容易被当成通往计划的过场。

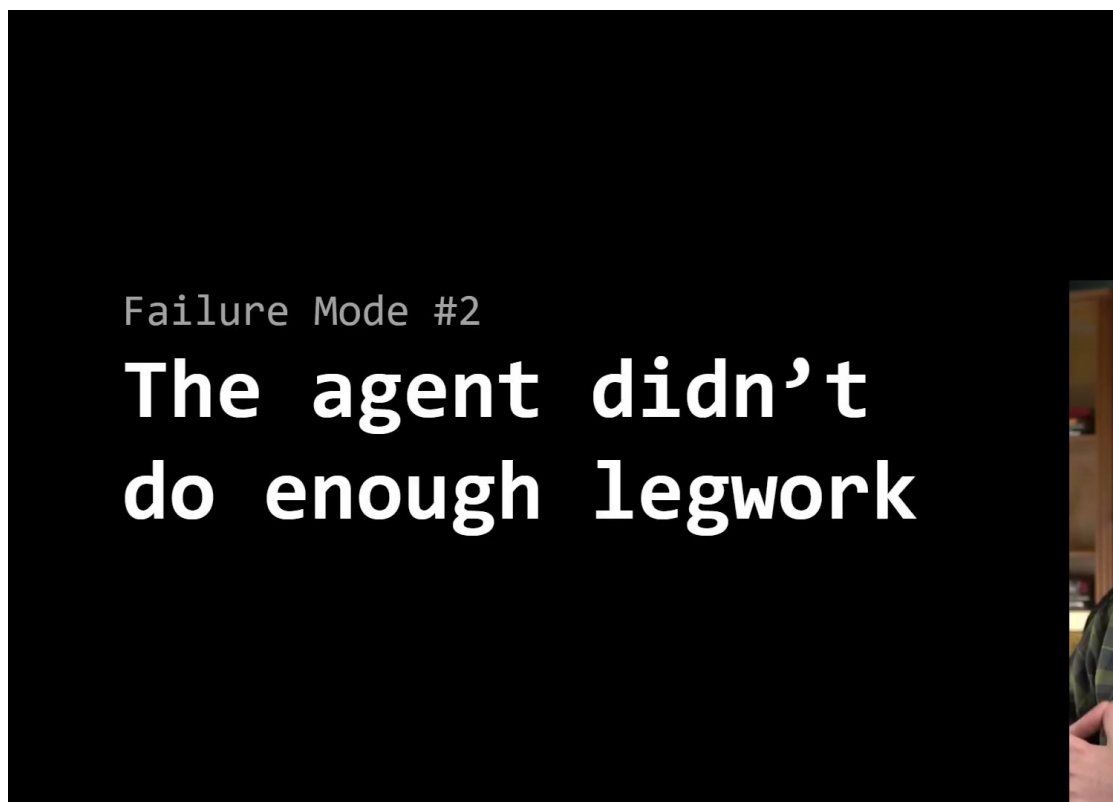


图 12: 失败模式：代理没有在当前步骤做足 legwork，尤其容易出现在澄清、探索、调研这些前置阶段。¹²

未知风险：可见终点会抢占注意力

一旦代理同时看到“澄清问题”和“创建计划”，后者会吸走注意力。人类会把澄清当作计划质量的前提；代理更容易把澄清当作计划前的最小手续。这就是阶段没有隔离时的注意力分配问题。

7.2 机制：phase isolation 与 attention allocation

作者的解决方案可以概括为两个概念：**phase isolation** 和 **attention allocation**。前者是阶段隔离，把不同性质的工作拆开，让代理一次只看到当前阶段；后者是注意力分配，让当前阶段在上下文中拥有足够高的优先级。

如果用一个简化模型表示代理对某步骤的投入：

$$E_{\text{current}} \propto \frac{A_{\text{current}}}{A_{\text{current}} + A_{\text{future}}}$$

其中：

- E_{current} 表示代理投入当前步骤的努力；
- A_{current} 表示当前步骤在上下文中的注意力权重；

¹²视频时间：00:14:55–00:15:10。

- A_{future} 表示未来交付物在上下文中的注意力权重。

当未来目标过早出现， A_{future} 上升，当前步骤获得的努力就会下降。作者的做法是降低未来步骤的可见度，让代理把精力压到眼前阶段。

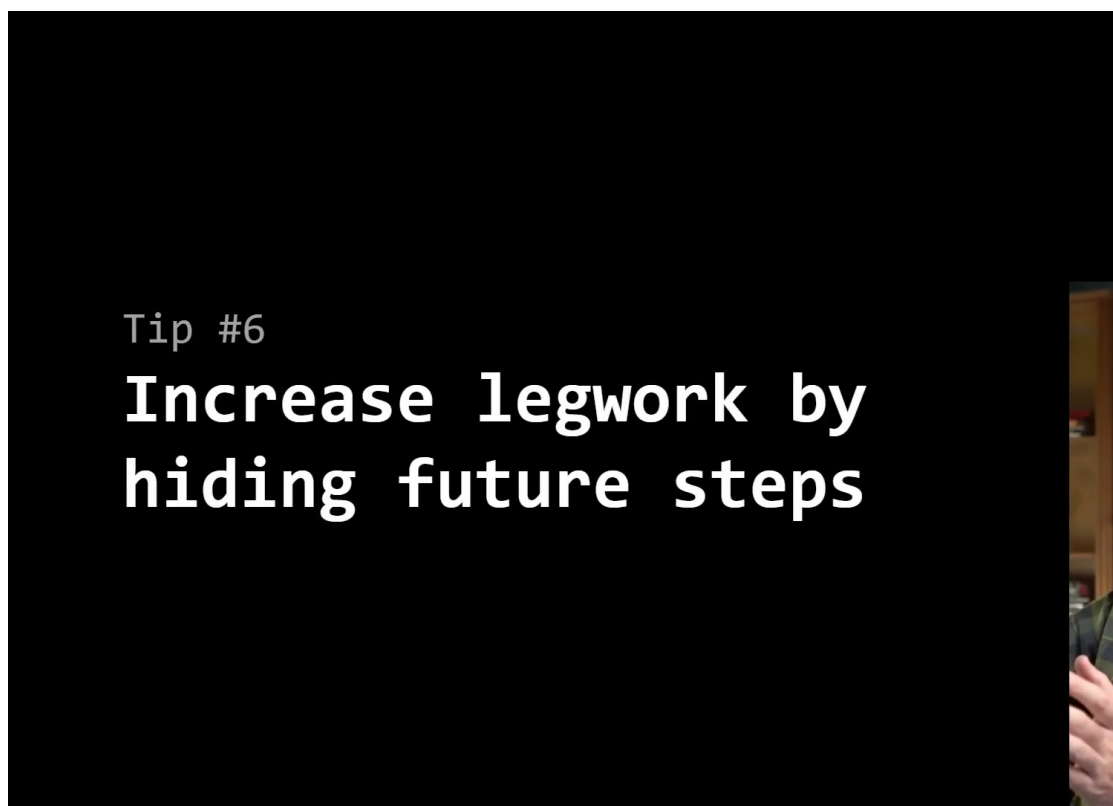


图 13: Tip #6: 通过隐藏未来步骤来增加当前阶段的 legwork。¹³

leg work 的核心规则

当某个前置阶段质量特别重要时，让代理只看见当前阶段。隐藏未来步骤的目的，是控制注意力预算：当前阶段有更完整的上下文、更明确的成功标准，也更难被后续交付物提前拉走。

7.3 grill-with-docs 与 to-prd：把计划前置工作拆成两个技能

作者用自己的 workflow 解释了这个机制。他没有把所有内容都塞进一个 Plan Mode。他把澄清和审问阶段拆成 grill-with-docs，让代理专门围绕资料、需求、文档和约束追问。这个技能完成后，再进入 to-prd，也就是写产品需求文档的阶段。

这相当于把一个“先澄清、再计划”的长流程拆成两个局部目标：

1. grill-with-docs：当前目标是把问题问透，把文档和上下文里的隐含约束挖出来；
2. to-prd：当前目标是把已经确认的信息整理成 PRD，并处理测试边界等结构化要求。

¹³ 视频时间：00:16:25–00:16:40。

这种拆分的价值在于，第一阶段不会被“赶紧生成 PRD”提前干扰。代理在 `grill-with-docs` 里只需要做追问、核对和压力测试；它没有看到完整的后续写作任务，因此更可能把前置工作做厚。等第一阶段产出足够信息，再把结果交给 `to-prd`，第二阶段就能基于更清楚的输入写出更可靠的文档。

和 vertical slice 的关系

`vertical slice` 通过一个行为锚词改变实现顺序；`leg work` 通过阶段隔离改变注意力分配。二者都属于 `steering`：前者改变代理怎样切任务，后者改变代理在当前阶段愿意投入多少工作。

7.4 什么时候值得隐藏未来步骤

隐藏未来步骤有成本。技能拆得越细，用户或编排者越需要负责阶段衔接。因此这个方法适合用在前置质量对最终结果影响很大的地方，例如需求澄清、代码库探索、安全审计、研究综述、测试策略确认。若某一步只需要机械执行，拆成独立技能的收益可能很低。

判断标准可以很朴素：如果代理经常急着进入后续交付物，导致当前阶段证据不足、问题没有问透、代码没有读完、风险没有列清，那么就值得把当前阶段单独隔离出来。隔离之后，当前阶段需要有自己的完成标准，例如“提出并追问关键歧义”“列出源码证据”“确认测试 seam”“给出可拒绝的假设清单”。没有完成标准的隔离，会变成另一种空泛提示。

7.5 本章小结

`leg work` 关注的是代理在当前阶段投入多少真实努力。作者用 `Plan Mode` 说明：当代理同时看到澄清问题和创建计划时，它往往会压缩澄清阶段。解决办法是阶段隔离，把 `grill-with-docs` 这类追问阶段和 `to-prd` 这类写作阶段拆开，让代理一次只面对一个目标。对技能设计者来说，这是一种注意力预算管理：未来步骤越早出现，当前步骤越容易被草草带过；当前阶段越清晰、越封闭、越有完成标准，代理就越可能做足 `leg work`。

8 Pruning：删除沉积物、杂质和无操作文本

8.1 为什么最后还要做 pruning

前面几章已经完成了技能设计的主体：先决定 `trigger`，再整理 `structure`，然后用 `leading words` 和 `leg work` 去改变代理的实际行为。到这里，技能已经能跑起来。问题也正是在这里出现：一个能跑起来的技能，很容易继续被追加说明、补充例外、塞入模板、修补边角，最后变成维护者自己也难以审阅的巨大文本块。

`pruning` 的任务就是在技能可用以后做维护性收缩。它关心的核心问题很朴素：哪些内容仍然改变行为，哪些内容只是在占用上下文、占用读者注意力、制造维护噪声。可以把它理解成整理一张长期多人共用的工作台。真正有用的工具要留在手边；只服务某个分支的工具要放到对应抽屉；过期便签和重复说明要从主工作面移走。这里讨论的是设计层面的剪枝和内容迁移，具体文件清理仍应遵守本地项目规则，把待清理对象移入待删除等暂存位置，由人最终确认。

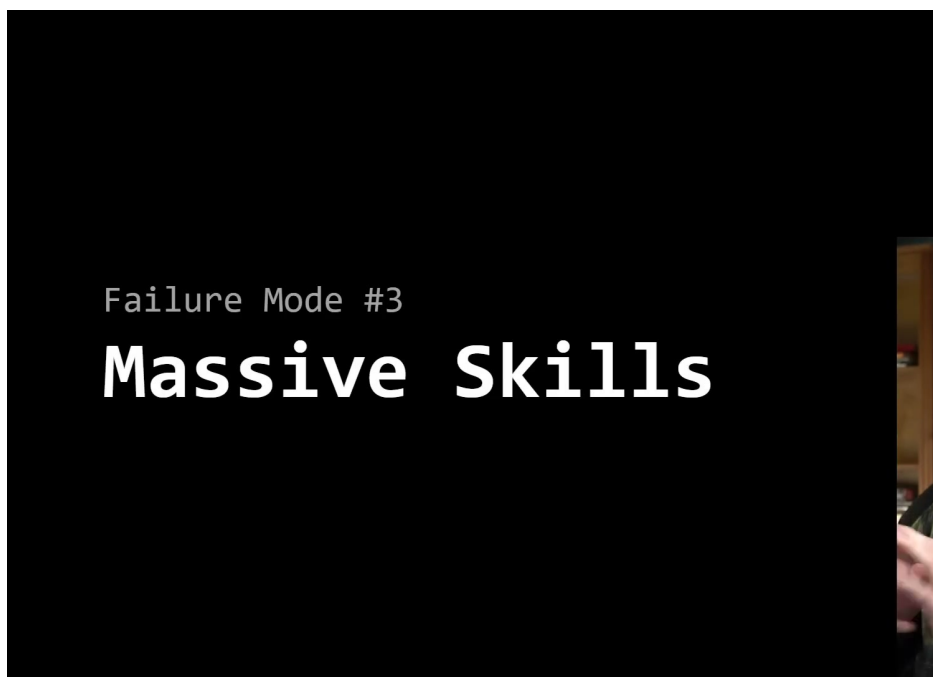


图 14: 演讲进入 pruning 段落时先指出第三类失败模式: Massive Skills。庞大的技能通常说明其它维护机制已经失效。¹⁴

Pruning 的核心纪律

pruning 超出文字润色范畴。它是一套技能维护纪律: 检查重复来源、检查分支归属、检查过期材料、检查 no-ops, 并把主技能文件压回“能改变代理行为”的最小形态。

这个步骤尤其重要, 因为技能会长期吸收团队的局部经验。每个人都可能为某次失败加上一段提醒; 每次问题复现时, 又有人补上一条规则。单条补丁看起来合理, 累计以后却会让技能失去形状。代理读取的是当前上下文里的文本分布, 维护者读取的是文件里的结构线索; 当这两者都被低价值内容淹没时, 技能就开始从操作规程退化成备忘录堆。

8.2 Massive skills: 巨大技能只是症状

演讲者把 Massive Skills 放在 pruning 的入口, 是因为“大”本身通常只是表层症状。真正的根因往往藏在三个地方: 重复定义、材料归属错误、无效指令堆积。巨大技能的危险也分三层。

- 对模型来说, 它增加上下文长度和注意力竞争。每多一段文字, 代理都要判断它是否当前相关。
- 对维护者来说, 它增加审计成本。维护者很难看清哪一段是规范, 哪一段只是历史补丁。
- 对团队来说, 它破坏共同语言。每个人都能继续追加内容, 却没有人愿意重排、移动、压缩旧内容。

这时最直接的第一步是检查 single source of truth。所谓 single source of truth, 就是一个定义、一个模板、一个流程判断只放在一个权威位置。比如 PRD template、test seam 的定义、

¹⁴视频画面时间区间: 00:16:55–00:17:10。

某个分支专用的输出格式，都不应在主流程、参考文件和示例里各写一遍。重复内容会带来两个坏结果：更新时漏改，或者不同位置逐渐长出细微差异。

Single source of truth 的直觉

可以把 `single source of truth` 想成公司里的唯一制度文件。若同一条报销规则同时写在员工手册、部门 Wiki、邮件模板和聊天置顶里，任何一次更新都会变成对齐问题。技能也是这样：一个概念若有多个来源，代理和维护者都会被迫猜测哪个版本才算准。

从技能结构看，`single source of truth` 还要求维护者区分“全局材料”和“分支材料”。全局材料服务所有路径，适合放在 `skill.md` 的主线上；分支材料只服务某个场景，适合放在外部参考文件，再由 `context pointer` 指向它。这样做并非追求目录漂亮，真实收益在于减少主路径的干扰项。

8.3 Sediment：多人协作留下的沉积物

`sediment` 的中文可以译作“沉积物”。这个词很准确：沉积物并不一定一开始就是垃圾，它常常来自真实工作流里的真实补充。问题在于它们一层层压上去以后，原有河道被抬高、变窄、变浑。共享 Markdown 文件最容易出现这种状态：每个人都敢加内容，较少有人敢改动别人写下的内容。



图 15: Sediment 指共享技能文件里逐层堆积的旧材料、局部补丁和归属不清的段落。它会让结构变浑，维护者也更难判断主线。¹⁵

沉积物通常有四种来源。

- 某次失败后的紧急补丁。它解决了当时的问题，后来却没有被整合进更稳定的步骤。
- 某个分支专用的说明。它只对少数任务有用，却停留在主技能路径里。

¹⁵ 视频画面时间区间：00:17:45–00:18:05。

- 过期的上下文。它曾经指向正确工具、模板或团队约定，后来项目已经迁移。
- 重复的解释。不同人用不同措辞解释同一个概念，让读者误以为存在多个规则。

沉积物的隐藏风险

sediment 最危险的地方在于它看起来常常“有点道理”。维护者如果只按句子质量判断，就会舍不得动它；更可靠的判断标准是结构归属和行为影响：这段话服务哪条分支，是否仍能改变代理下一步动作，是否已有权威位置承载同一信息。

处理沉积物时，第一动作应是回到 **structure**。先问：这段内容是否服务所有分支？若答案是否定的，就把它迁入对应分支的参考材料，或收进更合适的模板、示例、背景说明。若它已经过期或完全无关，则把它从主技能设计中剪出，并按项目安全流程放到待人工确认的位置。这样写，是为了强调维护纪律，同时避免把概念性 **pruning** 误读成随手执行破坏性文件操作。

8.4 Crud：杂质会遮住真正的操作意图

crud 可以译作“杂质”或“脏内容”。它和 **sediment** 有重叠，区别在于 **crud** 更强调低价值残留：半句旧解释、重复标题、含糊口号、失效链接、对当前任务没有帮助的上下文。沉积物像多年堆积的河床，杂质像工作台上散落的小纸片；每一片都不大，合在一起却会让人找不到刀、尺和图纸。

在技能里，**crud** 的典型表现包括：

- 把“请认真”“务必仔细”“高质量完成”这类空泛要求反复写入步骤。
- 在主文件里塞入只对某一次项目有用的历史背景。
- 对同一工具调用写多套近似说明，却没有说明何时采用哪一套。
- 把参考材料、范例、排错记录和主流程混成一段长文。

杂质的害处在于它降低信噪比。技能本质上是一份给代理执行的微型操作规程，文本里的每个段落都在争夺“下一步该做什么”的解释权。高信噪比的技能会让代理自然地沿着步骤走；低信噪比的技能则会让代理在大量礼貌性、背景性、历史性文字里猜重点。

一个实用类比

Python 项目里的 **README** 若同时包含安装步骤、旧版本迁移记录、作者碎碎念、过时截图和生产部署密钥说明，读者会很难判断第一步该做什么。技能文件也是面向执行的文档，它需要把“马上影响行为的内容”放在主路径，把“必要时才读取的内容”放进被指向的参考材料。

8.5 No-ops：看起来有意义，实际不改行为

no-ops 是演讲者特别强调的失败模式。这个词来自编程里的“no operation”，意思是执行以后没有实质效果的操作。放到技能里，**no-ops** 指那些看起来像指令，实际并不改变代理行为的文本。

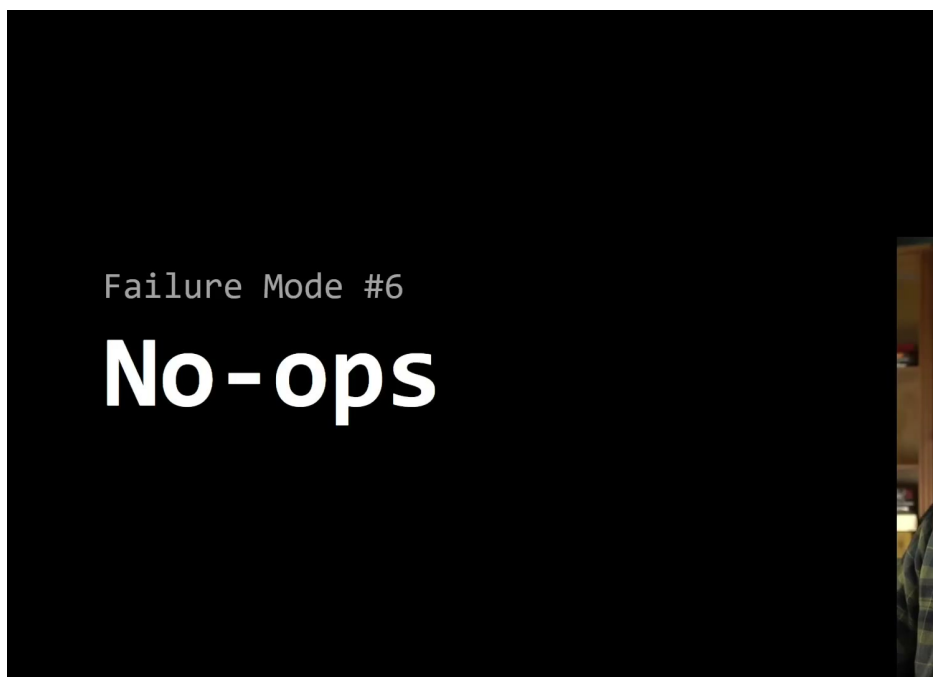


图 16: No-ops 是 pruning 段落中最常见、也最容易被忽视的失败模式：文字存在，行为没有随之改变。¹⁶

演讲者举了一个很典型的例子：一个 implementation skill 里有整段文字要求代理写一条长而详细的 commit message。问题在于，许多代理在没有这段话时也会写出还算完整的提交信息。于是这段话虽然占据空间，实际增量很小。它像在电梯按钮旁边贴一张“按按钮后电梯会来”的纸条；读者看到了文字，系统行为却没有因此改变。

No-op 的定义

no-op 指在当前技能上下文中无法产生可观察行为差异的说明。它可能语气强烈、措辞正确、看起来合理；判断标准仍然是行为差异，而非句子本身是否顺眼。

为了把这个判断做得更具体，可以用一个轻量公式表达“段落增量”：

$$\Delta B(p) = B(S) - B(S \setminus p)$$

- S 表示当前技能文本整体。
- p 表示正在审阅的某一段、某一句或某个参考块。
- $B(S)$ 表示代理在完整技能文本下的可观察行为。
- $B(S \setminus p)$ 表示审阅时假设 p 被移出主技能路径后的可观察行为。
- $\Delta B(p)$ 表示这段内容带来的行为差异。若差异接近零， p 很可能是 no-op。

这个公式属于审查启发式，它的价值在于迫使维护者追问：这段话到底让代理多做了什么、少做了什么、提前做了什么、延后做了什么？若回答只能停留在“它提醒代理要认真”，那就很危险。更好的文本应当给出可执行边界、具体检查项、失败条件、工具选择或输出格式。

¹⁶ 视频画面时间区间：00:18:45–00:19:00。

8.6 Deletion test: 用行为差异审查段落

`deletion test` 可以译作“删除测试”。在这里，它是一种概念审查方法：维护者假设把某段内容从主技能路径中移出，然后观察或推断代理行为是否发生实质变化。若没有变化，这段内容就应被压缩、移动、合并，或从主设计里剪出。

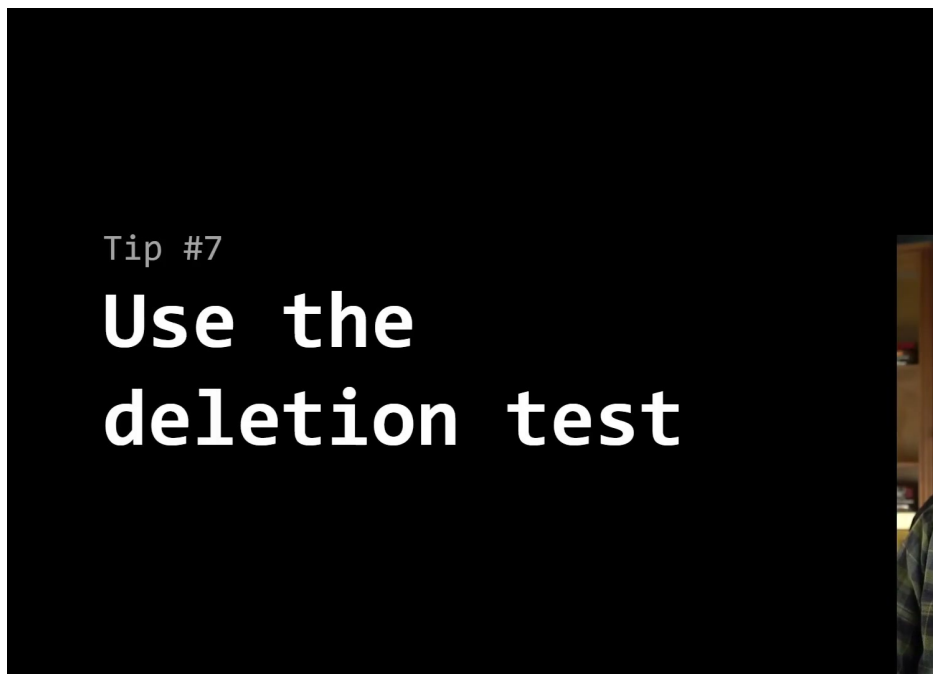


图 17: Deletion test 的问题很尖锐：如果某段话被移出主路径，代理行为是否真的变化？若没有变化，它很可能只是 no-op。¹⁷

一个可复用的审查流程如下。

1. 选中一个段落 p ，明确它声称要改变的行为。
2. 写下期望行为差异，例如“代理必须先问三个澄清问题”或“代理必须读取某个参考文件后再写计划”。
3. 假设 p 从主路径移出，判断代理是否仍会自然完成同一行为。
4. 若行为不变，优先把 p 压缩成一个更强的 `leading word`、并入已有步骤，或迁入分支参考材料。
5. 若行为明显变差，保留 p ，并把它写得更可观察、更可验证。

本地项目的清理边界

本章的 `deletion test` 属于技能设计审查，不能当成文件系统操作说明。若实际文件或目录需要退出当前工作面，应按本项目约定先移入当前输出目录下的 待删除，等待用户最终处置。

¹⁷ 视频画面时间区间：00:18:58–00:19:08。

这条边界非常重要。技能维护里说“prune out irrelevant stuff”，含义是从有效指令系统里剪掉无贡献内容；工程环境里处理文件时，还要遵守权限、审计和可恢复性规则。把两者混在一起，会让一个概念工具变成危险操作习惯。

8.7 本章小结

本章把 pruning 从“写完以后删一删”提升为技能维护纪律。巨大技能通常是症状，背后常见根因是重复来源、sediment、crud 和 no-ops。可靠的修剪顺序是：先恢复 single source of truth，再按分支移动材料，再处理过期和低价值内容，最后用 deletion test 检查每段文字是否真的改变代理行为。这样得到的技能更小、更清晰，也更容易被团队持续维护。

9 总结与延伸

9.1 演讲者的最后 checklist sweep

演讲收束时，演讲者把整套框架重新扫了一遍。这一段的价值在于把前面所有概念压回一个可执行的评审顺序：先看 trigger，再看 structure，再看 steering，最后做 pruning。读者如果只记住一件事，可以记住这条顺序，因为它正好对应技能从“何时进入上下文”到“如何保持长期可维护”的完整生命周期。

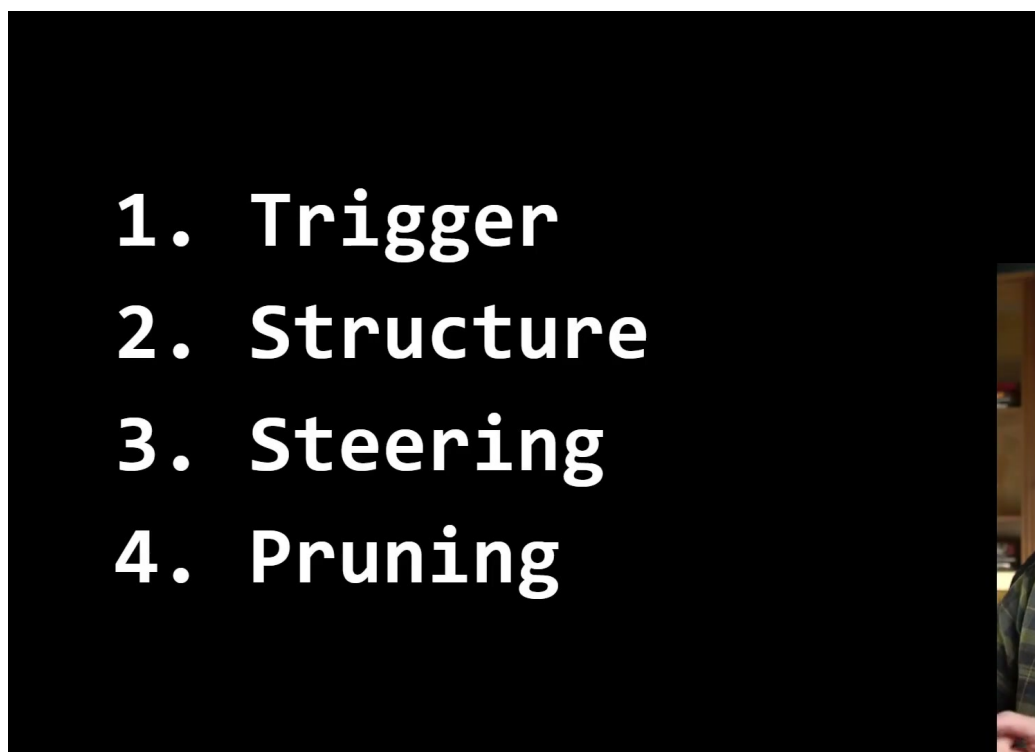


图 18: 最终回顾页把四个评审维度重新列出：Trigger、Structure、Steering、Pruning。它们共同构成一条从触发到维护的技能评审路线。¹⁸

¹⁸ 视频画面时间区间：00:19:15–00:19:30。

第一项是 **trigger**。评审者要确认技能是否在正确时间被调用，并明确它给谁增加负担。若技能是 **model-invoked skill**，它会增加模型的 **context load**，因为描述会常驻上下文并参与每次请求的判断。若技能是 **user-invoked skill**，它会增加用户的 **cognitive load**，因为用户必须记得何时手动触发。这里没有免费选择，只有负担在模型和用户之间如何分配。

第二项是 **structure**。评审者要检查技能是否分清 **steps** 和 **reference**，并检查分支材料是否仍塞在 **skill.md** 主路径里。一个稳定的技能主文件应该像机场指示牌：主路径清楚，分支入口明确，需要额外材料时再通过 **context pointer** 进入对应参考文件。

第三项是 **steering**。评审者要看技能是否把行为意图压缩成强有力的 **leading words**，并观察这些词是否会回到代理的推理轨迹和输出中。若一个词能反复把代理带回目标动作，它就是行为锚词；若一段话很长，却无法在代理行为中留下痕迹，它就需要被压缩或重写。

在 **steering** 这一项内部，还要检查一个子机制：**leg work**，也就是当前阶段是否做够工作。演讲者提醒，如果代理总是急着跳到后续目标，可以把技能拆成更窄的阶段，让代理一次只看到当前任务。这样做会增加当前步骤的注意力密度，尤其适用于澄清问题、代码库探索、需求访谈、设计审查这类容易被匆忙略过的工作。

第四项是 **pruning**。最终评审要扫过整个技能，寻找 **sediment**、**crud** 和 **no-ops**。这一步的目标是把技能压回小而有效的形态：重复内容归并到 **single source of truth**，分支内容移到分支材料，过期内容退出主设计，行为增量接近零的段落接受 **deletion test**。

最终 checklist 的一句话版本

好技能的评审顺序是：触发要准，结构要清，转向要有行为锚词，阶段要让代理做足 **leg work**，最后用 **pruning** 把沉积物、杂质和无操作文本清出主路径。

9.2 资源指针：writing great skills

演讲者最后给出的资源指针是 [mattpocock/skills](#) 仓库里的 **writing-great-skills**。这个链接具有操作入口意义；它说明本讲的框架已经被编码成一个可运行的技能。换句话说，演讲本身在示范一个递归动作：用“写好技能”的技能来检查和改进其它技能。

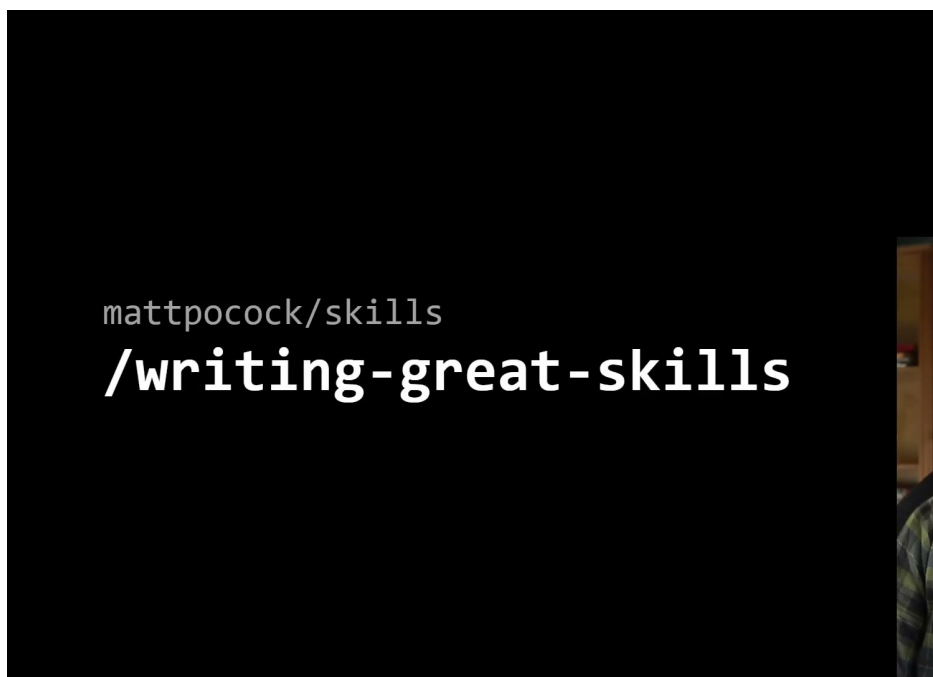


图 19: 演讲最后指向 `mattpocock/skills` 中的 `/writing-great-skills`。这给读者一个可操作入口：用同一套 checklist 审查自己的技能或社区技能。¹⁹

这个资源的使用方式可以分成三类。第一，用它写新技能：从一开始就决定触发方式、结构边界、引导词和剪枝规则。第二，用它审查旧技能：拿现有 `skill.md` 逐项跑 checklist，找出上下文负载、结构混乱和行为无效段落。第三，用它筛选社区技能：下载别人写的技能前，先检查它是否有清晰触发、可导航结构、具体行为锚词和维护纪律。

为什么资源指针重要

一个好框架如果只能停留在演讲里，影响力会很有限。把框架做成技能，相当于把抽象评审标准变成可复用操作流程。读者不需要每次重新解释“好技能是什么”，可以让代理按同一套语言和同一套检查顺序执行审查。

演讲结尾还提到 AI coding crash course 和 `aihero.dev` newsletter。这些信息对课程背景有帮助，但对本 PDF 的技术主线贡献较小。真正应该保留的是 `writing great skills` 资源，因为它直接承接本讲的核心框架，也让读者能把 checklist 立刻应用到自己的技能库里。

9.3 作者提炼：把 checklist 变成评审流程

从 PDF 作者角度看，本讲的核心贡献超出零散写作建议，形成了一套小型操作规程设计法。技能是给代理读取、遵循、复述、调用工具、生成输出的执行上下文。因此，评审技能时需要同时追问“文字是否清楚”“这段文字如何进入上下文”“它如何改变行为”“它如何长期保持干净”。

可以把整套方法压缩成四步评审流程，其中 `leg work` 是 `steering` 内部的一条关键检查线。

1. **Trigger: 定义进入条件。**技能由用户触发还是模型触发？它增加的是 `context load`、`cognitive load`，还是调用不确定性？这一项决定技能进入系统的方式。

¹⁹ 视频画面时间区间：00:20:05–00:20:20。

2. **Structure: 建立导航结构。**主路径只保留通用步骤和必要定义；分支材料通过 `context pointer` 指向外部参考。这样可以 `skill.md` 更接近索引和操作手册，远离资料仓库式堆放。
3. **Steering: 写入行为锚词并隔离高投入阶段。**用 `leading words` 把复杂行为压缩成可重复、可回响的词；对容易被跳过的工作，例如澄清问题、资料查证、代码库探索，用阶段隔离让代理只面对当前阶段的完成标准。前者处理行为方向，后者处理 `leg work`。
4. **Pruning: 做定期复审。**查重复、查分支归属、查过期材料、查 `no-ops`。用 `deletion test` 追问每段文字的行为增量，并按项目规则把退出主路径的内容放到可恢复、可审计的位置。

本讲的压缩结论

`writing great skills` 的本质是把技能当作代理的微型操作规程：有精确触发条件，有可导航结构，有能进入推理轨迹的行为词，有让当前阶段做足工作的边界，还有可重复的 `pruning` 复审流程。

这个结论也解释了为什么“写得更详细”常常无法解决技能质量问题。技能里的文字越多，代理可选择的解释路径越多；若没有结构和行为锚词，更多文字只会制造更多解释空间。真正有效的详细度，应当出现在正确位置：主路径给出步骤和判断，参考材料承载模板和背景，分支材料只在需要时进入上下文。

9.4 四项主框架之间的内在关系

这套 `checklist` 的四项主框架形成了一条因果链；`leg work` 属于 `steering` 的阶段隔离子机制。

- `trigger` 决定技能是否进入上下文。没有进入条件，其它设计都没有执行机会。
- `structure` 决定进入以后代理读到什么。结构越清，代理更容易找到当前步骤。
- `steering` 决定代理如何理解动作方向。强 `leading words` 会把意图压成可复用的行为语言；阶段隔离会保护 `leg work`，避免代理提前奔向最后交付物。
- `pruning` 决定技能能否长期维持前三项。没有 `pruning`，旧补丁会逐渐稀释触发、结构、转向和阶段边界。

这种关系可以用一个简单表达式记住：

$$Q_{\text{skill}} \approx T \times S \times St(W, L) \times P$$

- Q_{skill} 表示技能质量的综合结果。
- T 表示 `trigger` 的准确性。
- S 表示 `structure` 的可导航性。
- $St(W, L)$ 表示 `steering` 的综合强度，其中 W 是 `leading words` 的行为锚定能力， L 是 `leg work` 的充分程度。

- P 表示 pruning 的持续执行程度。

乘法只是教学比喻，意思是任何一项接近零都会拖垮整体。一个触发错误的技能写得再精美也不会被正确调用；一个结构混乱的技能有再好的引导词也可能被淹没；一个从未 pruning 的技能会慢慢吞掉原本清晰的设计。

读者容易漏掉的未知风险

组织采用技能时，最容易高估“下载更多技能”的收益，低估“维护技能”的成本。技能库一旦进入多人协作阶段，就会出现版本漂移、重复定义、局部补丁和上下文负载膨胀。没有评审流程的技能库，迟早会把最初的效率优势转化成新的操作风险。

9.5 把这套方法用于真实团队

真实团队可以把本讲框架落成一个周期性评审节奏。每个技能上线前，先过一次准入检查：触发方式是否明确，主路径是否短，关键术语是否有 `single source of truth`，高风险步骤是否有明确停顿点。技能上线后，再按固定周期做维护检查：有没有新沉积物，是否出现重复参考，是否有段落无法通过 `deletion test`。

对工程团队来说，还可以把技能评审接入代码评审习惯。审查者不只看语法和格式，还要问四类问题：这段内容会让代理多做什么？它是否属于所有分支？它是否已有权威来源？它是否可以被一个更强的 `leading word` 替代？这些问题比“写得详细一点”更有效，因为它们直接面向行为。

对个人使用者来说，最实用的起点是小规模清点。先挑三个最常用技能，分别标出 `trigger`、`steps`、`reference`、`leading words` 和 `pruning` 候选段落。这个动作会立刻暴露技能是否真的可导航。若连使用者自己都无法标出这些结构，代理更难稳定遵循。

9.6 本章小结

本章完成了演讲的收束：演讲者最后用 `Trigger`、`Structure`、`Steering`、`Pruning` 做 checklist sweep，并把读者指向 `mattpocock/skills` 里的 `writing-great-skills`。PDF 作者的结构化提炼是：好技能有明确触发、清晰结构、强行为锚词与足够 `leg work` 共同支撑的 `steering`，以及持续 `pruning`。把这四项做成重复评审流程，技能才会从一次性提示文本变成可维护的代理操作规程。